RHp Tag Report

Dongjun Park, Seungmin Jeon, Donghyun Kim

April 4, 2025

Abstract

In 6D pose estimation, various approaches are employed, including deep learning-based methods, traditional computer vision techniques, and fiducial marker systems. We opted for fiducial markers due to their balance of simplicity, accuracy, and real-time performance. Specifically, Apriltag was chosen for its high detection accuracy, robustness, and consistent performance across various conditions. These capabilities are largely due to its evolution, which has incorporated advancements like hash-based decoding and adaptive preprocessing, thereby enhancing detection confidence and efficiency. We conducted experiments to evaluate tag detection performance under various conditions - distance, angle, and lighting conditions - focusing on the tag with a size of 0.04x0.04 meters. Our goal was to identify the tag with the highest detection reliability. We found that detection performance is influenced by both external factors and the type of tag. These findings will aid in selecting the most suitable AprilTag and environmental settings for specific scenarios in future applications. By understanding how different factors affect detection performance, we can optimize the use of Apriltag, ensuring reliable and efficient 6D pose estimation.

1 Introduction

Estimating the 6D pose of objects is a fundamental problem in robotics, essential for tasks such as manipulation, navigation, and interaction with the environment. In our project, we explored multiple approaches to allow a robot to estimate the 6D pose, including:

- **Deep learning-based methods**: Using RGB or RGB-D sensor, the models (ex.view point matching model[1], regression model[2], 2D-3D matching[3] and pose ranking model[4]) trained on large datasets. These methods are trained on large datasets, which can include RGB-D images or videos, to learn features that are crucial for pose estimation. Deep learning models perform well in complex environments, handling partial occlusions and noisy conditions effectively, while also adapting to various lighting conditions and environmental changes. They require powerful GPUs and large amounts of memory, which can be costly and resource intensive. This makes them challenging to deploy in applications where computational resources are limited.
- **Traditional computer vision techniques**: Relying on geometric features, keypoints, and Perspective-n-Point (PnP) algorithms. These methods typically involve detecting keypoints or features in an image and matching them to a 3D model of the object. The PnP algorithm is then used to compute the pose from these correspondences. This process often begins with keypoint detection, where algorithms like SIFT (Scale-Invariant Feature Transform) are used to identify distinctive points such as corners or edges. These keypoints are then matched to a 3D model, establishing correspondences between the 2D image features and their 3D counterparts. These techniques are often more interpretable and can be more efficient than deep learning methods, as they provide clear insight into how the model makes decisions and require less computational resources. However, they may struggle with complex scenes or objects lacking distinct features, where feature extraction and matching become challenging.

- **Fiducial marker systems**: Using predefined patterns like Apriltag or ArUco for efficient pose detection. These markers are placed in the scene or on objects, allowing for easy detection and pose estimation. Their high contrast makes them robust in simple environments, enabling them to be easily distinguished from backgrounds. Additionally, fiducial markers are straightforward to implement with minimal setup and do not require complex algorithms or deep learning models. This simplicity makes them highly accessible and user-friendly, as they can be easily printed and attached to objects, providing immediate functionality. The limitations include physical constraints that require ideal flat surfaces, limited information capacity due to spatial resolution, and vulnerability to light changes and distortions.

Given the limited computational resources of embedded systems commonly used in robotics, we prioritized lightweight and efficient methods. Among the available options, fiducial markers offered a favorable balance between simplicity, accuracy, and real-time performance.

Table 1: Benchmark comparison of fiducial marker systems for pose estimation [5]

Feature	**ARTag**	**AprilTag**	**ArUco**	**STag**
Detection Accuracy	Medium	High	Medium	High
Detection Robustness	Low	High	Medium	High
Detection Range	Short	Long	Medium	Long
Detection Angle Tolerance	Low	High	Medium	High
Processing Time / Speed	Medium	Medium-High	High	Medium
Frame Rate Support	Medium	Real-time	Real-time	Real-time
Lighting Robustness	Low	High	Medium	High
Translation Accuracy	Medium	High	Medium	High
Rotation Accuracy	Medium	High	Medium	High
Pose Stability / Jitter	Low	High	Medium	High
Noise Resilience	Low	High	Medium	High
Marker Dictionary Size	Low	High	High	Medium
Marker Size Flexibility	Limited	Flexible	Flexible	Flexible
Error Correction Capability	Basic	Strong	Medium	Strong
Marker Type	Binary	Binary (v3)	Binary	Hybrid (gradient-based)
Planar / Non-Planar Support	Planar	Both	Both	Both
Open Source Availability	No	Yes	Yes	Yes
Cross-Platform Support	Limited	Wide	Wide	Wide
Ease of Integration	Low	High	High	Medium
Hardware Requirements	Medium	Low	Low	Medium

Based on the benchmark results summarized in Table 1 from Kalaitzakis et al. [5], we selected **Apriltag** as the primary tool to estimate pose. Although ArUco is fast and well integrated with OpenCV, and STag offers high robustness and hybrid detection, Apriltag provided the best overall balance between numerous features. In detail, apriltag demonstrates exceptional detection accuracy and robust performance in various conditions. Its high tolerance to detection angles ensures reliable marker recognition, even when viewed from oblique or extreme perspectives, while its strong lighting robustness allows it to function effectively under diverse illumination settings. Moreover, Apriltag offers superior translation and rotation accuracy, enabling precise estimation of a marker's position and orientation. This level of precision is essential for robotics systems that require dependable spatial understanding for navigation and manipulation tasks. Additionally, Apriltag supports real-time frame rates, ensuring efficient processing of visual data and rapid responsiveness to dynamic environments. Its low hardware requirements and open-source availability make it highly accessible and easy to integrate without the need for specialized equipment.

Although the ultimate objective of our work is to enable object recognition and pose estimation without reliance on markers, our initial experiments using Apriltag provided a reliable foundation for development and evaluation. This marker-based approach allowed us to validate our system pipeline and gain insights that will guide future work in markerless object recognition.

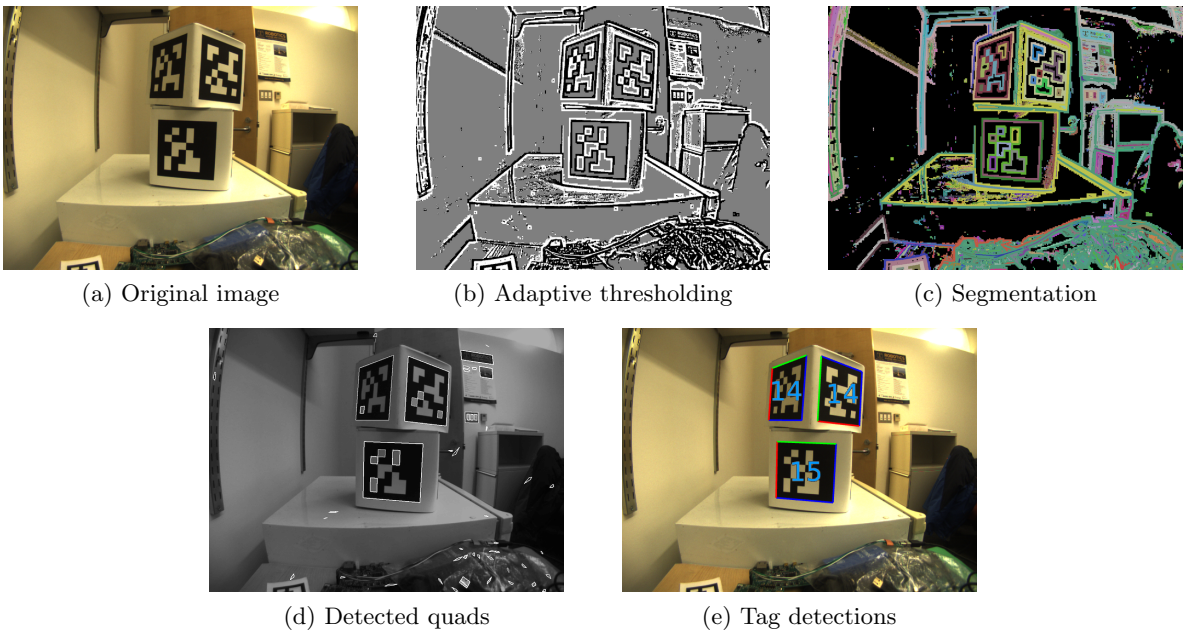
2 Method

The AprilTag system is a visual fiducial framework designed for efficient and robust tag detection and localization. This section describes the principles, components, and evolution of AprilTag across its three generations—the original AprilTag system [6], AprilTag 2 [7], and AprilTag 3 [8]. The code we used to detect and localize AprilTags is available at https://github.com/AprilRobotics/apriltag.

2.1 Detection Pipeline

AprilTag detects 2D fiducial markers using a multi-stage pipeline optimized for both speed and robustness. Each stage progressively refines image data into precise tag detections. The corrected and detailed pipeline steps are as follows (see Figure 1):

Figure 1: Intermediate steps of the AprilTag detector



(a) Original image (b) Adaptive thresholding (c) Segmentation

(d) Detected quads (e) Tag detections

1. **Image Preprocessing**

   The detection process starts with the original RGB or grayscale image (Figure 1a). If the input is RGB, the system converts the image to grayscale.

   AprilTag 2 and later apply adaptive thresholding (Figure 1b), partitioning the grayscale image into small tiles (e.g., 4×4 pixels), and computing local maxima and minima to create a binary image robust to varying illumination conditions:

$$I_b(x,y) = \begin{cases} 1 & \text{if } I_g(x,y) > T_{local}(x,y) \\ 0 & \text{otherwise} \end{cases} \tag{1}$$

   where $T_{local}(x,y)$ is the locally computed threshold.

2. **Connected Component Analysis**

   The binarized image undergoes segmentation into connected regions (Figure 1c) using a Union-Find algorithm (see pseudocode in Algorithm 1). Each pixel (x,y) is assigned a label $l(x,y)$ indicating its connected component. The Union-Find structure merges neighboring pixels with the same binary value:

$$\text{if } I_b(x,y) = I_b(x',y') \Rightarrow \text{Union}((x,y),(x',y')) \tag{2}$$

$$l(x,y) = \text{Find}(x,y) \tag{3}$$

AprilTag 3 improves boundary extraction by clustering edge pixels into continuous boundaries:

$$\mathcal{B}_{r_0,r_1} = \left\{ \left(\frac{x+x'}{2}, \frac{y+y'}{2} \right) \mid I_b(x,y) \neq I_b(x',y'), \text{Find}(x,y) = r_0, \text{Find}(x',y') = r_1 \right\} \tag{4}$$

where $\mathcal{B}$ represents boundary pixels.

3. **Quad Detection**

   Quad candidates (quadrilaterals) are detected by fitting polygons to clustered boundary edges (Figure 1d). AprilTag 3 employs Principal Component Analysis (PCA) for accurate line fitting. Let boundary points $\{p_i\}$ cluster around a line represented by the unit vector $\mathbf{u}$ and centroid $\bar{p}$. The PCA fitting minimizes:

$$\min_{\|\mathbf{u}\|=1} \sum_i \left\| (p_i - \bar{p}) - ((p_i - \bar{p}) \cdot \mathbf{u})\, \mathbf{u} \right\|^2 \tag{5}$$

   Detected quads undergo geometric verification, checking angle regularity and convexity.

4. **Homography Estimation**

   For each candidate quad, a homography H mapping canonical tag coordinates (x_i^0, y_i^0) to observed image coordinates (x_i^1, y_i^1) is computed by solving:

$$w_i \begin{bmatrix} x_i^1 \\ y_i^1 \\ 1 \end{bmatrix} = H \begin{bmatrix} x_i^0 \\ y_i^0 \\ 1 \end{bmatrix}, \quad i = 1,2,3,4 \tag{6}$$

   Solving this system provides a perspective transformation for each quad.

5. **Bit Sampling and Enhancement**

   Using the homography H, a grid corresponding to the tag's payload is sampled from the image. AprilTag 3 enhances sampling accuracy with bilinear interpolation and Laplacian sharpening kernel K:

$$K = I + 0.25 \begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix} \tag{7}$$

   This operation accentuates bit contrast, improving robustness, particularly for smaller tags.

6. **Tag Validation and Decoding**

   The sampled bits undergo decoding through error-correcting code techniques such as Hamming distance d. A valid tag minimizes the decoding error (Figure 1e):

$$\hat{t} = \arg\min_{t \in \mathcal{T}} \text{Hamming}(b_s, b_t) \tag{8}$$

   where b_s is the sampled bit vector and b_t is the stored bit vector for valid tags. Tags are validated if the minimum Hamming distance is below a threshold defined by the error-correcting capability.

7. **Pose Estimation**

   Upon validation, the final 6-DOF pose of the tag relative to the camera is computed. Given intrinsic camera parameters K and homography H, the pose is recovered by decomposing H:

$$H = K \begin{bmatrix} \mathbf{r}_1 & \mathbf{r}_2 & \mathbf{t} \end{bmatrix} \tag{9}$$

   where r_1,r_2 are rotation vectors and t is translation. The third rotation vector r_3 is obtained by the cross product:

$$\mathbf{r}_3 = \mathbf{r}_1 \times \mathbf{r}_2 \tag{10}$$

   The rotation matrix R and translation vector T fully define the tag's pose:

$$R = [\mathbf{r}_1 \quad \mathbf{r}_2 \quad \mathbf{r}_3], \quad T = \mathbf{t} \tag{11}$$

   This provides robust real-time localization suitable for robotics and augmented reality.

Algorithm 1 Find Boundaries using Union-Find

```
function FindBoundaries(im, w, h)
    // Find connected components using union-find
    uf ← UnionFind(w · h)
    for all pixel (x, y) do
        for all neighbor (x', y') do
            if im[x, y] = im[x', y'] then
                uf.union(y · w + x, y' · w + x')
            end if
        end for
    end for
    // Group pixels which form a continuous boundary
    h ← HashTable()
    for all pixel (x, y) do
        for all neighbor (x', y') do
            if im[x, y] ≠ im[x', y'] then
                r_0 ← uf.find(y · w + x)
                r_1 ← uf.find(y' · w + x')
                id ← ConcatenateBits(Sort(r_0, r_1))
                if id ∉ h then
                    h[id] ← List()
                end if
                p ← ((x+x')/2, (y+y')/2)
                h[id].append(p)
            end if
        end for
    end for
end function
```

2.2 Tag Coding and Error Correction

AprilTag utilizes lexicographic codes to ensure a minimum Hamming distance d between all valid tags across their four possible rotations. The Hamming distance between two bit sequences b_1 and b_2 is defined as:

$$\text{Hamming}(b_1, b_2) = \sum_i |b_1[i] - b_2[i]| \tag{12}$$

where each bit $b[i]$ is either 0 or 1. AprilTag 1 introduced a rectangle complexity heuristic to mitigate false positives by assessing the complexity of bit patterns within tags. Specifically, complexity C was calculated by counting bit transitions along rows and columns:

$$C = \sum_{i,j} \left(|b_{i,j} - b_{i,j+1}| + |b_{i,j} - b_{i+1,j}| \right) \tag{13}$$

AprilTag 3 improved upon this approach by integrating the Ising energy metric, which evaluates the visual complexity and asymmetry of tags:

$$E = -\sum_{\langle i,j \rangle} b_i b_j \tag{14}$$

where the sum is taken over neighboring bit pairs $\langle i, j \rangle$. Tags with lower Ising energy E exhibit higher visual complexity, reducing confusion with natural background textures.

Table 2: False positive rates by bit error correction level

Bits corrected	**36h10 FP (%)**	**36h15 FP (%)**
0	0.000001	0.000000
1	0.000041	0.000002
2	0.000744	0.000029
3	0.008714	0.000341
4	0.074459	0.002912
5	0.495232	0.019370
6	N/A	0.104403
7	N/A	0.468827

AprilTag 2 accelerated the decoding process by precomputing and storing all tag codes within 2-bit errors in a hash table, reducing lookup complexity from $O(n)$ to $O(1)$. AprilTag 3 introduced early rejection based on quad contrast thresholds before decoding attempts, significantly enhancing performance in low-resolution or noisy scenarios.

2.3 Flexible Layouts (AprilTag 3)

AprilTag 3 introduced flexible layout strings, enabling the design of non-rectangular, circular, annular, and recursive tag structures (Figure 2). Each layout cell is defined using specific labels: black ('b'), white ('w'), data ('d'), or ignored ('x'). These layout strings must maintain fourfold rotational symmetry and include a clearly defined detection border to facilitate robust identification.

Formally, a tag layout L can be described as a two-dimensional matrix:

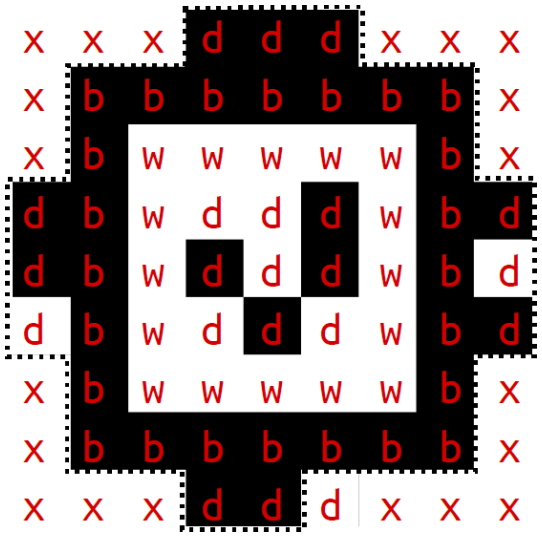
$$L = \{l_{i,j}\}, \quad l_{i,j} \in \{\text{'b'}, \text{'w'}, \text{'d'}, \text{'x'}\} \tag{15}$$

The symmetry condition requires:

$$l_{i,j} = l_{N+1-j,i} = l_{N+1-i,N+1-j} = l_{j,N+1-i} \tag{16}$$

where N denotes the tag layout dimension. By strategically placing data cells outside the detection border, payload density is optimized without increasing the tag's physical dimensions.

Figure 2: Layout string overlay for a 21h7 circular tag



This approach facilitates innovative use cases such as long-range unmanned aerial vehicle (UAV) tracking with nested recursive tags, significantly enhancing detectability and reliability in challenging visual environments.

2.4 Summary

AprilTag has evolved considerably, beginning as a gradient-based quad detector (original AprilTag) and advancing through hash-based accelerated decoding (AprilTag 2), ultimately resulting in a robust, flexible, and low-latency fiducial marker system (AprilTag 3). This latest generation integrates generalized layout designs, sophisticated complexity metrics, and adaptive preprocessing techniques.

Each generational improvement has significantly boosted detection confidence, reduced false positives, and optimized computational efficiency. Performance metrics such as detection confidence and computational latency were rigorously measured under various conditions. For each scenario, the average detection confidence and latency were computed over multiple trials to provide a reliable performance summary.

This comprehensive evolution positions AprilTag as a leading fiducial marker system for diverse real-time robotic and augmented reality applications, from powerful computational systems to resource-constrained embedded platforms.

3 Experiment and Result

We confirm the performance of tag detection under various conditions through a series of experiments involving multiple tags. Our objective is to identify the tag that achieves the highest detection reliability, specifically for a target scenario with a detection tag size of 0.04x0.04 (m). We designed experiments to evaluate the effects of three primary factors and evaluation metric is decision margin. The evaluation metric's code is described in algorithm 2. All experiments were conducted on a system running Ubuntu 22.04 LTS.

Algorithm 2 Find Decision Margin

function FINDDECISIONMARGIN($family, im, quad, entry, im\_samples$)
  // Initialize variables
  $white\_score \leftarrow 0$, $black\_score \leftarrow 0$
  $white\_score\_count \leftarrow 1$, $black\_score\_count \leftarrow 1$
  $rcode \leftarrow 0$
  // Step 1: Sample white and black borders to build grayscale models
  $whitemodel \leftarrow$ GrayModel(), $blackmodel \leftarrow$ GrayModel()
  for all pattern in predefined border patterns **do**
    $is\_white \leftarrow pattern[4]$
    for $i \leftarrow 0$ to $2 \cdot family.black\_border + family.d - 1$ **do**
      $(tagx, tagy) \leftarrow$ NormalizeCoordinates($pattern, i, family$)
      $(px, py) \leftarrow$ HomographyProject($quad.H, tagx, tagy$)
      if (px, py) within im bounds **then**
        $v \leftarrow im[py, px]$
        if is_white **then**
          $whitemodel$.add($tagx, tagy, v$)
        else
          $blackmodel$.add($tagx, tagy, v$)
        end if
      end if
    end for
  end for
  $whitemodel$.solve(), $blackmodel$.solve()
  // Step 2: Validate threshold
  if $whitemodel$.interpolate$(0,0) - blackmodel$.interpolate$(0,0) < 0$ **then**
    return -1
  end if
  // Step 3: Decode bits and compute decision margin
  for $bitidx \leftarrow 0$ to $family.d \cdot family.d - 1$ **do**
    $(bitx, bity) \leftarrow (bitidx \mod family.d, \lfloor bitidx / family.d \rfloor)$
    $(tagx, tagy) \leftarrow$ NormalizeBitCoordinates($bitx, bity, family$)
    $(px, py) \leftarrow$ HomographyProject($quad.H, tagx, tagy$)
    if (px, py) within im bounds **then**
      $v \leftarrow im[py, px]$
      $thresh \leftarrow \frac{blackmodel.\text{interpolate}(tagx,tagy) + whitemodel.\text{interpolate}(tagx,tagy)}{2}$
      $rcode \leftarrow rcode \ll 1$
      if $v > thresh$ **then**
        $white\_score \leftarrow white\_score + (v - thresh)$
        $white\_score\_count \leftarrow white\_score\_count + 1$
        $rcode \leftarrow rcode \vee 1$
      else
        $black\_score \leftarrow black\_score + (thresh - v)$
        $black\_score\_count \leftarrow black\_score\_count + 1$
      end if
      if $im\_samples$ is provided **then**
        $im\_samples[py, px] \leftarrow (1 - (rcode \wedge 1)) \cdot 255$
      end if
    end if
  end for
  // Step 4: Finalize tag code and return decision margin
  $entry \leftarrow$ QuickDecodeCodeword($family, rcode$)
  return $\min\left(\frac{white\_score}{white\_score\_count}, \frac{black\_score}{black\_score\_count}\right)$
end function

3.1 Distance

In this experiment, we evaluated the detection performance of three AprilTag families—16h5, 25h9, and 36h11—by varying the tag-to-detector distance from 0.3 m to 0.8 m in 0.1 m increments. Each tag measured 4 cm × 4 cm, and detection confidence was assessed using the average decision margin.

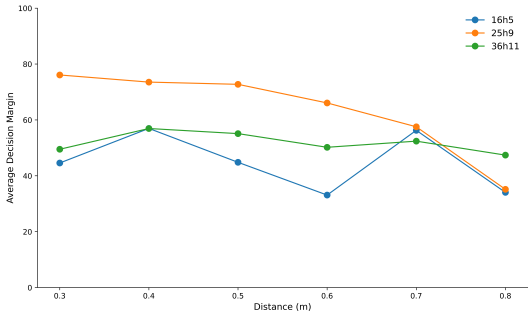


Figure 3: Distance experiment result

Figure 3 shows the results, with 16h5, 25h9, and 36h11 represented by blue, orange, and green lines, respectively. The decision margin decreases with distance for all families, as expected due to reduced pattern resolution. At 0.3 m, 25h9 has the highest margin (around 80), followed by 36h11 (around 50) and 16h5 (around 45). At 0.8 m, 36h11 leads with a margin of approximately 50, followed by 25h9 (around 40) and 16h5 (around 40).

The 16h5 family (4x4 bit pattern, Hamming distance 5) generally shows the lowest margin due to its simpler pattern, but an unexpected spike occurs at 0.7 m, rising from 30 at 0.6 m to 50. This may stem from optimal lighting or tag orientation at that distance.

The 25h9 family (5x5 bit pattern, Hamming distance 9) performs consistently well, starting at 80 at 0.3 m and decreasing to 50 at 0.8 m. Its larger cell size (0.8 cm per cell) compared to 36h11 (0.67 cm per cell) aids detection at greater distances. The 36h11 family (6x6 bit pattern, Hamming distance 11) was expected to perform best due to its higher error correction capability but underperforms compared to 25h9 at most distances, except at 0.8 m where it peaks at 50. The small 4 cm x 4 cm tag size likely limits 36h11's performance, as its finer cells are harder to resolve with increasing distance. Using a larger tag or higher-resolution camera could improve 36h11's detection.

3.2 Angle Experiment

We conducted an angle experiment to evaluate the detection performance of three AprilTag families—16h5, 25h9, and 36h11—under varying angular conditions. The tag-to-detector distance was fixed at 0.3 m, and the tags, each measuring 4 cm x 4 cm, were rotated around the x-axis at angles of 0°, 15°, and 30°. The coordinate system was defined with the forward direction as the x-axis, the left direction as the y-axis, and the upward direction as the z-axis. All tags were observed downward using the same robotic camera to ensure consistent detection conditions across the tests.

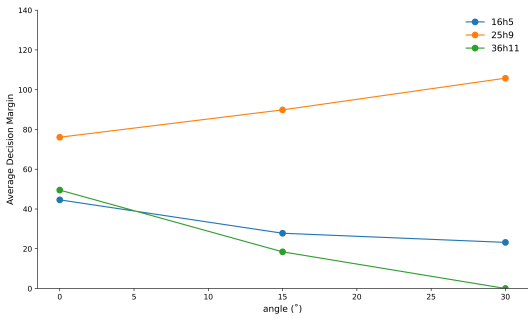


Figure 4: Angle experiment result

Figure 4 presents the results of the angle experiment, with 16h5, 25h9, and 36h11 represented by blue, orange, and green lines, respectively. At 0°, 25h9 exhibits the highest average decision margin (around 80), followed by 36h11 (around 50) and 16h5 (around 40). As the angle increases to 30°, 25h9 shows an unexpected increase in decision margin, rising to nearly 110. In contrast, 16h5 and 36h11 experience a decline, with their decision margins dropping to approximately 30 and 5, respectively, indicating a reduced detection confidence at larger angles.

Several factors may contribute to the observed variations in detection performance. The 25h9 tag, with its 5×5 bit pattern and Hamming distance of 9, features a cell size of 0.8 cm, providing a balanced complexity that enhances its robustness against perspective distortion at larger angles. In contrast, the 16h5 tag (4×4, 1 cm per cell) has a simpler structure, making it more susceptible to noise, while the 36h11 tag (6×6, 0.67 cm per cell) has smaller cells that become harder to resolve at wider angles.

The interaction between the camera's field of view and the tag's orientation may also influence detection results. At 15° and 30° angles, the 25h9 pattern may align more effectively with the camera's perspective, improving its visibility. In contrast, the smaller cells of 36h11 could become less distinguishable, while the simpler structure of 16h5 may amplify noise sensitivity.

Additionally, lighting and reflection effects could play a role. As the angle increases, surface reflections might shift, potentially enhancing contrast for 25h9 while degrading it for the other tags. Another possible explanation is that the hyperparameter settings of the detection algorithm may have been optimized in a way that favors 25h9, affecting its performance relative to the other tag families.

3.3 Light Conditions

We evaluated the impact of lighting on the detection performance of AprilTag families—16h5, 25h9, and 36h11—under controlled illumination levels. The tag-to-detector distance was fixed at 0.3 m, and the tags, each measuring 4 cm x 4 cm, were observed using a robotic camera. Illumination was adjusted using a desk lamp and measured with a lux meter application on a smartphone, with all ambient light sources except the desk lamp blocked to isolate external light. The illumination levels tested were approximately 400 lux, 750 lux, 900 lux, 1100 lux, and 1800 lux, allowing us to assess the effect of lighting variations on detection performance.

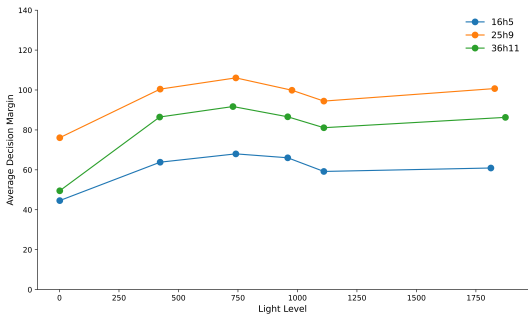


Figure 5: Light experiment result

The light experiment can see at Figure 5 and revealed a non-linear relationship between illumination levels and AprilTag detection performance, as indicated by the decision margin. Initially, as light increased from 0 to 750 lux, the decision margin for all tag families improved significantly. This is likely because sufficient illumination allows the camera to clearly capture the tag features, leading to more accurate and confident detection. The peak performance around 750 lux suggests an optimal lighting condition where tag features are well-defined and easily distinguishable.

However, a slight decrease in the decision margin was observed between 750 and 1100 lux. This temporary dip could be attributed to potential overexposure or glare at these intermediate higher light levels. Overexposure can saturate the camera sensor, washing out the fine details of the tag patterns and making them harder to differentiate.

Beyond 1100 lux, the decision margins began to recover or stabilize. This recovery might be due to the camera's automatic exposure adjustments compensating for the increased brightness, or simply that even with potentially high overall brightness, the contrast within the tag pattern itself remains sufficient for reliable detection, especially for the more complex families like 25h9 and 36h11 which have more internal features. Overall, the results suggest that while some illumination is crucial for good detection, excessively high levels might initially hinder performance before the system can adapt or the inherent tag robustness allows for recovery. The consistent high performance around 750 lux indicates a generally favorable illumination range for these AprilTag families under the given experimental setup.

4 Conclusion

Our experiments evaluated the detection reliability of three AprilTag families—16h5, 25h9, and 36h11—each with a tag size of 0.04m $\times$ 0.04m, under varying conditions including distance, rotation angle, and lighting. The results showed that the 25h9 family generally performed best overall, particularly in maintaining high detection rates as the angle increased. This robustness can be attributed to factors such as tag complexity, camera orientation, and algorithmic hyperparameters. Additionally, detection performance improved with increasing light levels up to 750 lux, beyond which overexposure likely caused a decline.

These findings highlight the significant impact of both environmental conditions and the choice of AprilTag family on detection performance. This insight is crucial for optimizing AprilTag configurations in robotics applications, ensuring reliable and efficient use for tasks such as pose estimation and object tracking. By selecting appropriate tag sizes and configurations tailored to specific operational conditions, users can enhance the overall effectiveness of their systems.

References

[1] Wadim Kehl, Fabian Manhardt, Federico Tombari, Slobodan Ilic, and Nassir Navab. Ssd-6d: Making rgb-based 3d detection and 6d pose estimation great again. In *Proceedings of the IEEE international conference on computer vision*, pages 1521–1529, 2017.

[2] Yu Xiang, Tanner Schmidt, Venkatraman Narayanan, and Dieter Fox. Posecnn: A convolutional neural network for 6d object pose estimation in cluttered scenes. *arXiv preprint arXiv:1711.00199*, 2017.

[3] Chen Wang, Danfei Xu, Yuke Zhu, Roberto Martín-Martín, Cewu Lu, Li Fei-Fei, and Silvio Savarese. Densefusion: 6d object pose estimation by iterative dense fusion. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 3343–3352, 2019.

[4] Bowen Wen, Wei Yang, Jan Kautz, and Stan Birchfield. Foundationpose: Unified 6d pose estimation and tracking of novel objects. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 17868–17879, 2024.

[5] Michail Kalaitzakis, Brennan Cain, Sabrina Carroll, Anand Ambrosi, Camden Whitehead, and Nikolaos Vitzilaios. Fiducial markers for pose estimation. *Journal of Intelligent & Robotic Systems*, 101(4):71, March 2021.

[6] Edwin Olson. AprilTag: A robust and flexible visual fiducial system. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pages 3400–3407. IEEE, May 2011.

[7] John Wang and Edwin Olson. AprilTag 2: Efficient and robust fiducial detection. In *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, October 2016.

[8] Maximilian Krogius, Acshi Haggenmiller, and Edwin Olson. Flexible layouts for fiducial tags. In *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, October 2019.